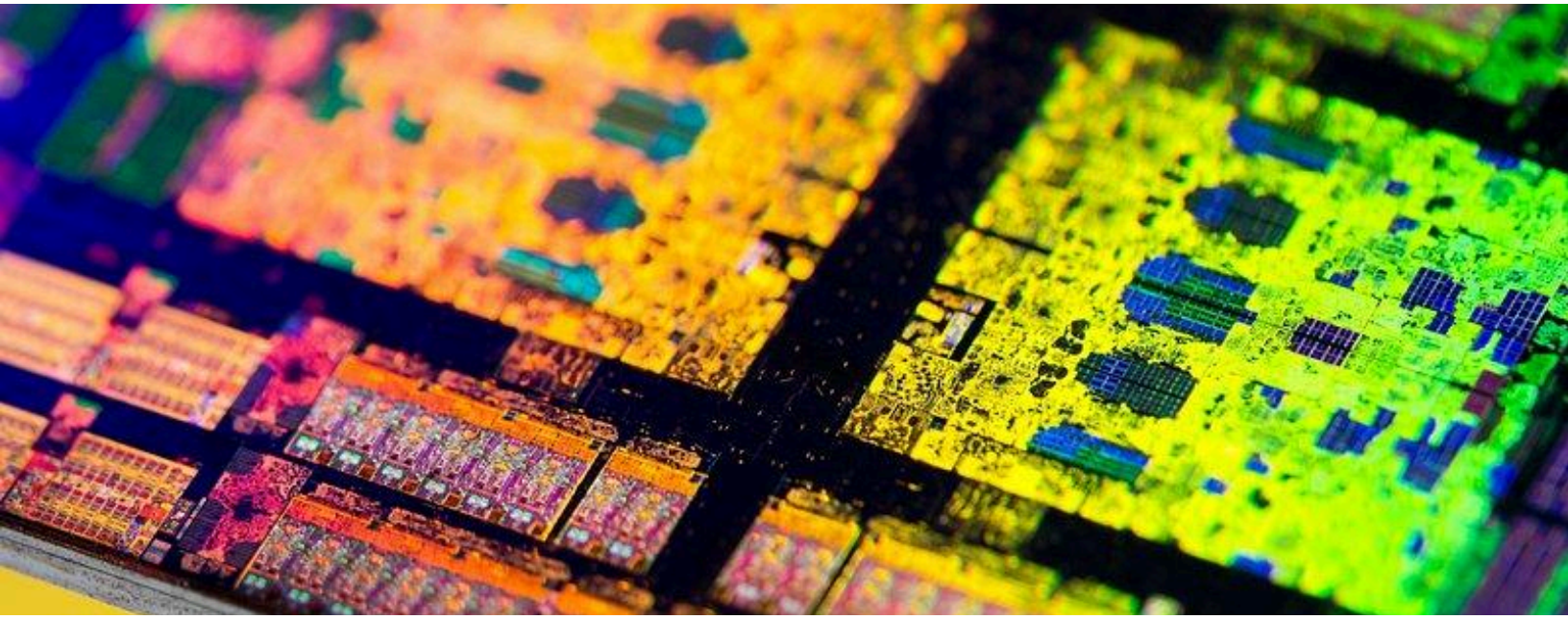
ADDV

Assignment 1



Vaibhav Singh

2022555

Tarush Garg

2022537

Output

```
✦ ~/function/addv/assigment_1
❯ ./sim.out

        SystemC 3.0.1-Accellera --- Sep 16 2025 11:52:16
        Copyright (c) 1996-2024 by all Contributors,
        ALL RIGHTS RESERVED
================ Simulation Start ================
CPU executed: add 4 36 -> result 40 at 10 ns
CPU executed: eq 49 55 -> result 0 at 14 ns
CPU executed: rem 20 2 -> result 0 at 29 ns
CPU: Acknowledged write(100) to 0xfff0 at 29 ns
CPU executed: sub 9 12 -> result fffffffd at 40 ns
CPU: Acknowledged write(15) to 0xff00 at 79 ns
CPU executed: add 12 1000 -> result 3f4 at 89 ns
CPU executed: rem 5 8 -> result 5 at 104 ns
Memory: completed write(64) to 0xfff0 at 79 ns
CPU: received BEGIN_RESP (write completed) at 79 ns
CPU: Acknowledged write(15) to 0xff40 at 129 ns
CPU executed: rem 100 7 -> result 2 at 144 ns
CPU executed: sub 125 25 -> result 64 at 155 ns
CPU executed: eq 47 47 -> result 1 at 159 ns
CPU executed: add 5 6 -> result b at 169 ns
Memory: completed write(15) to 0xff00 at 129 ns
CPU: received BEGIN_RESP (write completed) at 129 ns
CPU: Acknowledged write(15) to 0xff80 at 179 ns
Memory: completed write(15) to 0xff40 at 179 ns
CPU: received BEGIN_RESP (write completed) at 179 ns
Memory: completed write(15) to 0xff80 at 229 ns
CPU: received BEGIN_RESP (write completed) at 229 ns

================ Simulation End ================
[SW] Total simulated time: 229 ns
================================================

✦ ~/function/addv/assigment_1
❯
 [4] [1:zsh]  2:nvim>MacBook-VS.local [17-09-2025][23:48]
```

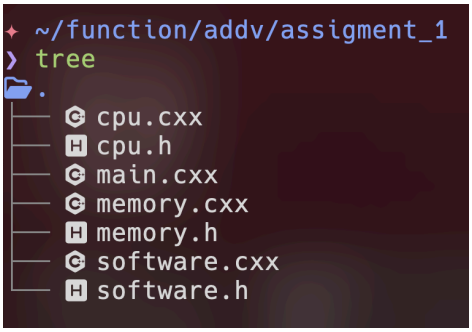
How to execute

We have created a bash function that will call c++ compiler and include all the lib file and inc file

To run the code

Compile the main.cxx and run the executable

Code Explanation



There are different files for each block 3 in blocks in total, and a main file that calls othe files.

Main

This file is the entry point and top-level structure for your entire simulation. Its primary job is to set up the environment, create the components, and start the simulation.

Instantiation: Inside the ***sc_main*** function, it creates one instance of each of your modules: Software, CPU, and Memory.

Execution: It calls ***sc_start***, which kicks off the SystemC scheduler and begins running the simulation processes defined in the other modules.

Software

This module acts as the testbench or stimulus generator for the simulation. It doesn't model a piece of hardware but rather provides the inputs needed to test the system.

Command List: It contains a ***std::vector*** of strings, where each string is a command to be executed (e.g., "add 4 36", "write 100 0xfff0").

It loops through the command list, packages each command string into a **TLM payload**, and sends it to the CPU using a blocking transport call ***b_transport***

CPU

This module models a Central Processing Unit. It acts as an intermediary, receiving commands from the software and either processing them internally or forwarding them to memory.

Dual Sockets: It has two sockets: a target socket to receive commands from the Software and an initiator socket to send commands to the Memory.

Command Parsing: Its main function, ***b_transport***, receives the payload from the software, extracts the command string, and parses it to understand the operation (e.g., add, write) and the data.

For Write Operations: The CPU does not handle the write. Instead, it creates a new memory-specific TLM payload and forwards the request to the Memory module using ***nb_transport_fw*** (non-blocking transport). It doesn't add any delay itself for this action.

The logical part for the CPU to handle the arithmetic calculations i also defined in this file.

Memory

The **Memory** module models a simple storage component with built-in latency, designed for transaction-level communication. It acts as a passive target device, responding to incoming requests from the CPU.

- **Target Socket**:
  The module exposes a single TLM target socket (`tlm_utils::simple_target_socket`), which allows it to receive read and write requests from initiator components such as the CPU.
- **Storage**:
  Internally, the memory uses a `std::map<uint32_t, int>` to represent its addressable storage space. The address acts as the key, while the associated value represents the stored data. This flexible structure allows sparse memory allocation.
- **Latency Modeling**:
  The `nb_transport_fw` function introduces a configurable latency to mimic realistic memory delays. Instead of executing requests instantly, the module adds a delay (e.g., `sc_time(50, SC_NS)`) to simulate access time. This delay is communicated back to the initiator, ensuring that the CPU observes the correct timing behavior.
- **Behavior**:

- On **write requests**, the memory stores the incoming value at the specified address.
- On **read requests**, it retrieves the value from the address if present (or returns a default if not initialized).
- The module acknowledges every valid transaction with the modeled delay.

- **Role in System**:
  The Memory module does not initiate transactions itself; it only responds to requests from initiators. This makes it a reactive component, accurately reflecting the passive nature of hardware memory blocks.